

PRD: SP/SSP Licensing Agent (WhatsApp Bot)

Product Requirements Document — Draft v1

1. Problem Statement

SSPs/sellers manually review a customer's existing Microsoft license estate and build renewal/upgrade quotes (ME3, ME5, ME7 + Copilot) in spreadsheets. This is slow, error-prone, and inconsistent across sellers. This product automates that workflow as a conversational WhatsApp bot: the seller uploads the customer's license CSV, picks a commercial scenario, edits it, and gets a priced, presentable comparison in minutes.

2. Goals

- Let an SSP/seller upload a customer's license export (CSV) via WhatsApp and get an instant, clean visual breakdown of the current estate.
- Generate accurate pricing for 4 scenarios: Renew As-Is, ME3+Copilot, ME5+Copilot, ME7, using a maintained rate card as the pricing source of truth.
- Let the seller edit quantities/SKUs and override migration decisions, with pricing recalculating live.
- Produce a final side-by-side scenario comparison (table + PDF) sendable straight back over WhatsApp.
- Keep migration/classification logic deterministic and centralized so all 4 scenarios stay consistent with each other.

3. Non-Goals (MVP)

- No multi-tenant support, single SSP org only. Multi-tenant is a future consideration.
- No customer-facing self-serve access, seller/SSP-only tool.
- No persistent storage of the customer's raw uploaded CSV processed in-memory per session only.
- No support for non-Microsoft product lines in MVP.
- No e-signature/contract execution, output is a commercial recommendation, not a binding order.

4. Users

- Primary: SSP/Seller — uploads license files, picks scenarios, edits configs, sends final quote to their own record.
- Secondary (future): Sales manager / approver reviewing seller-generated quotes — not in MVP scope.

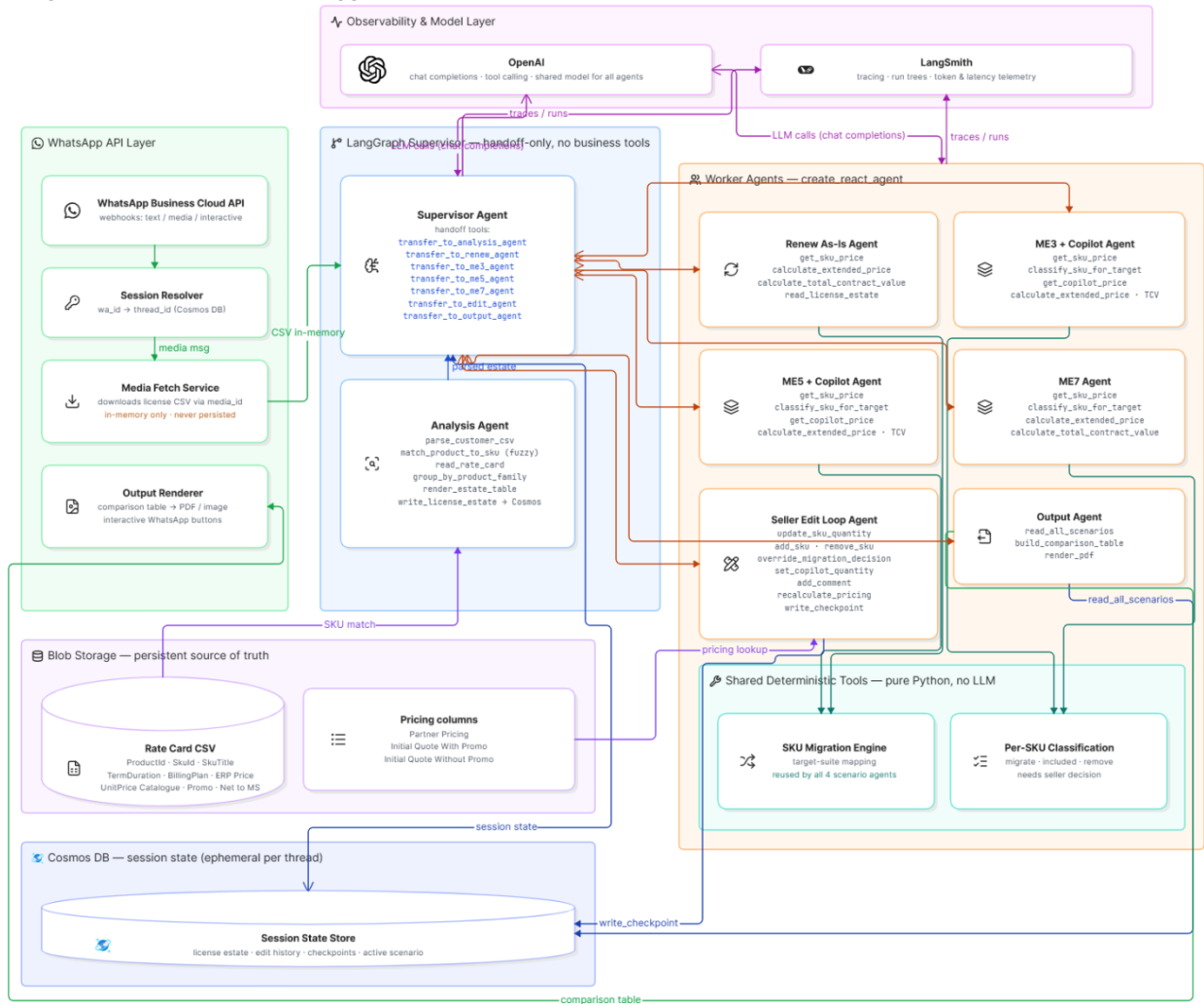
5. Confirmed Tech Stack

Layer	Choice
API server	FastAPI (Python)
Messaging channel	WhatsApp Cloud API (Meta) — requires Meta App + WhatsApp Business Platform registration
Session/state store	Azure Cosmos DB
Rate card storage	Azure Blob Storage (native)
LLM	OpenAI
Agent orchestration	LangGraph — langgraph-supervisor (supervisor + handoff pattern)
Observability	LangSmith
Hosting	Azure App Service
Auth (seller access)	WhatsApp number allowlist only (MVP)
Tenancy	Single-tenant (one SSP org)
MVP scenario scope	All 4 scenarios (Renew, ME3+Copilot, ME5+Copilot, ME7) at launch

6. Architecture Overview

Reference diagram (source of truth, editable):

SP/SSP Licensing Agent — LangGraph Supervisor + Handoff (WhatsApp)



eraser

Architecture

Flow

1. Seller sends license CSV as a WhatsApp media message.
2. WhatsApp Cloud API webhook → FastAPI receives it → Session Resolver maps wa_id → thread_id (Cosmos DB).
3. Media Fetch Service downloads the file (in-memory, not written to Blob).
4. analysis_agent parses the CSV, matches Product Title against the Blob rate card's SkuTitle (fuzzy match), groups by product family, and writes the normalized estate to Cosmos DB.
5. supervisor agent (handoff-only, no business tools) asks the seller which scenario to build, then hands off to the matching scenario agent.

6. Scenario agent (renew_agent / me3_agent / me5_agent / me7_agent) reads the estate, calls the shared deterministic SKU Migration Engine + Per-SKU Classification tools, pulls pricing from the Blob rate card, and returns a priced configuration.
7. Seller can hand off to edit_agent at any point to change quantities/SKUs/Copilot count/override migration — every edit triggers a pricing recalculation and a Cosmos DB checkpoint write.
8. output_agent builds the final side-by-side comparison table across all requested scenarios and renders it to PDF/image.
9. WhatsApp Output Renderer sends the final comparison back to the seller as a message + file.

All LLM calls (Supervisor + every worker agent) go through Azure OpenAI. Every graph run is traced in LangSmith for debugging and QA.

7. Agent Design (LangGraph)

7.1 Supervisor (create_supervisor)

No business tools — only handoff tools, auto-generated: transfer_to_analysis_agent, transfer_to_renew_agent, transfer_to_me3_agent, transfer_to_me5_agent, transfer_to_me7_agent, transfer_to_edit_agent, transfer_to_output_agent.

Routing behavior defined via prompt= — this flow is mostly deterministic (upload → analyze → choose scenario → edit → finalize), so the supervisor's prompt should encode that sequence explicitly.

7.2 analysis_agent

- parse_customer_csv(file_bytes) -> list[RawLicenseRow]
- match_product_to_sku(product_title) -> {product_id, sku_id, confidence}
- read_rate_card() -> DataFrame (Blob Storage)
- group_by_product_family(skus) -> dict
- render_estate_table(estate) -> visual_table
- write_license_estate(thread_id, estate) (Cosmos DB)

7.3 renew_agent / me3_agent / me5_agent / me7_agent

- get_sku_price(product_id, sku_id, term, billing_plan) — Blob rate card
- classify_sku_for_target(sku, target_suite) → shared SKU Migration Engine
- calculate_extended_price(qty, unit_price)
- calculate_total_contract_value(line_items)
- get_copilot_price(qty) — ME3/ME5; conditional for ME7
- read_license_estate(thread_id) (Cosmos DB)

7.4 edit_agent (Seller Edit Loop)

- update_sku_quantity, add_sku, remove_sku, override_migration_decision, set_copilot_quantity, add_comment
- recalculate_pricing(thread_id)
- write_checkpoint(thread_id, state) (Cosmos DB)

7.5 output_agent

- read_all_scenarios(thread_id)
- build_comparison_table(scenarios)
- render_pdf(comparison_json)

7.6 Shared deterministic tools

Plain Python functions, not LLM calls — reused by all 4 scenario agents:

- SKU Migration Engine — target-suite mapping table
- Per-SKU Classification — outputs migrate | included | remove | needs_decision per SKU

Keeping these as pure functions (not agent judgment) is what guarantees ME3/ME5/ME7 stay consistent with each other on the same input estate.

8. Data Model

8.1 Cosmos DB (session state — ephemeral per customer thread)

- sessions container: wa_id, thread_id, conversations, created_at, last_active
- license_estates container: thread_id, parsed/normalized SKU list, source = "whatsapp_csv"

Raw customer CSV is never persisted, only the parsed/normalized result.

8.2 Blob Storage (rate card — persistent source of truth)

CSV columns: ProductId, Skuld, SkuTitle, TermDuration, BillingPlan, ERP Price, UnitPrice/Catalogue, Promo (If new to MS), Net to MS, Expected Partner Pricing with Promo, Expected Partner Pricing, Initial Quote With Promo, Initial Quote Without Promo.

8.3 Customer CSV (uploaded via WhatsApp — in-memory only)

Confirmed columns so far: Product Title, Total Licenses, Expired Licenses, Assigned Licenses.

Open — need remaining columns: expiry/renewal date field, any SKU/part-number identifier, unit cost if present.

9. Non-Functional Requirements

- Latency: end-to-end response (upload → estate table) target < 15s for typical CSVs (~50–200 rows).
- Reliability: Cosmos DB checkpointing means a dropped WhatsApp session can resume mid-flow without losing edits.
- Observability: every agent run traced in LangSmith, tagged by thread_id.
- Security: rate card and customer data never leave Azure tenant boundary; raw customer CSV not persisted; WhatsApp allowlist enforced at ingress.
- Cost control: use a cost-appropriate Azure OpenAI model tier for deterministic steps; reserve larger models for judgment-heavy steps (scenario selection, ambiguous SKU matching).

10. Rollout Plan

10. Phase 0 — Infra setup: Meta app registration + WhatsApp number provisioning, Azure App Service + Cosmos DB + Blob Storage provisioning, LangSmith project setup.
11. Phase 1 — Core pipeline: WhatsApp webhook → CSV intake → Analysis Agent → estate table rendering (no pricing yet).
12. Phase 2 — Pricing engine: Blob rate card integration, SKU Migration Engine, Per-SKU Classification, all 4 scenario agents.
13. Phase 3 — Edit loop + Supervisor wiring: full LangGraph supervisor/handoff graph, seller edit tools, checkpointing.
14. Phase 4 — Output & comparison: Output Agent, PDF rendering, final WhatsApp delivery.
15. Phase 5 — Hardening: allowlist auth, error handling for malformed CSVs/unmatched SKUs, LangSmith-driven QA pass.

11. Open Questions / Items Still Needed

16. Remaining customer CSV columns — expiry date field name, any SKU identifier, unit cost (if present).
17. OpenAI model deployment — which model (e.g. GPT-4o / GPT-4.1 family) and region/deployment name?
18. Rate card update cadence — who updates the Blob CSV, how often, and does the agent need to detect staleness?
19. Fuzzy-match confidence threshold — when match_product_to_sku confidence is low, should the bot ask the seller to confirm, or auto-flag as "needs decision"?
20. CI/CD & repo — greenfield repo or existing one? Preferred CI (GitHub Actions / Azure DevOps)?
21. Copilot licensing rules for ME7 — is Copilot bundled or separately priced? (Currently marked conditional — needs a definitive rule.)

12. Setup & Access Requirements

Full checklist of accounts, resources, and credentials needed to build and run this product, split by service. Dev and prod are called out separately where the setup differs.

12.1 Azure Blob Storage

- Storage account created, container for the rate card CSV (e.g. rate-cards).
- Prod: prefer Managed Identity on the App Service with “Storage Blob Data Reader” role instead of a raw key.
- Dev: connection string is fine locally; Azurite emulator also works to avoid a live resource.

Credentials needed:

```
AZURE_STORAGE_ACCOUNT_NAME
AZURE_STORAGE_CONNECTION_STRING (or AZURE_STORAGE_ACCOUNT_KEY)
```

12.2 Meta / WhatsApp Business Platform

- Meta Business Manager account (business verification required for prod, not for dev/sandbox).
- Meta Developer app created at developers.facebook.com → add the “WhatsApp” product.
- Dev: Meta provides a free test number + test recipient list (up to 5 numbers) automatically — no business verification needed to start building.
- Prod: a real verified business phone number added to WhatsApp Manager, business verification completed, message templates approved for anything outside the 24h session window.
- Webhook needs a public HTTPS URL — ngrok / Azure dev tunnel for local dev, the real App Service URL for prod.

Credentials needed:

WHATSAPP_ACCESS_TOKEN	(temp 24h token in dev, permanent System User token in prod)
PHONE_NUMBER_ID	
WABA_ID	(WhatsApp Business Account ID)
META_APP_SECRET	(webhook signature verification)
VERIFY_TOKEN	(arbitrary string you set for webhook handshake)

12.3 OpenAI

- OpenAI platform account with billing enabled, API key generated from platform.openai.com.
- Model selected for each agent role (e.g. GPT-4o for judgment-heavy steps like scenario selection and ambiguous SKU matching, a smaller/cheaper model for deterministic steps if desired).
- Dev vs prod: same API key with usage tracked by project/tag, or fully separate API keys per environment for cost isolation and rate-limit separation, recommended to use separate keys so a dev bug can't burn prod quota.

Credentials needed:

OPENAI_API_KEY
OPENAI_MODEL

12.4 Azure Cosmos DB

- Cosmos DB account (SQL/Core API), database + containers: sessions, license_estates, scenarios.
- Dev: Cosmos DB Emulator can run locally to avoid cloud cost during development.
- Prod: prefer Managed Identity + RBAC over a raw key where possible.

Credentials needed:

COSMOS_DB_URI
COSMOS_DB_KEY

12.5 Azure App Service

- App Service Plan + Web App (Linux, Python runtime) created.
- Deployment method decided: GitHub Actions / Azure DevOps pipeline / az webapp deploy.
- Custom domain + TLS if a branded webhook URL is wanted instead of *.azurewebsites.net.

Credentials needed:

Publish profile OR federated GitHub OIDC credential (for CI/CD)
App Settings (env vars) for every credential listed in this section

12.6 LangSmith

- LangSmith account + workspace, project created (e.g. sp-ssp-licensing-agent).
- Dev and prod should use separate projects so test traces don't pollute prod debugging.

Credentials needed:

```
LANGSMITH_API_KEY  
LANGSMITH_PROJECT  
LANGCHAIN_TRACING_V2=true
```

12.7 Consolidated .env (local dev)

```
WHATSAPP_ACCESS_TOKEN=  
PHONE_NUMBER_ID=  
WABA_ID=  
META_APP_SECRET=  
VERIFY_TOKEN=  
  
OPENAI_API_KEY=  
OPENAI_MODEL=  
  
AZURE_STORAGE_CONNECTION_STRING=  
  
COSMOS_DB_URI=  
COSMOS_DB_KEY=  
  
LANGSMITH_API_KEY=  
LANGSMITH_PROJECT=  
LANGCHAIN_TRACING_V2=true
```

This PRD will be updated as the above open items are resolved. Next step once confirmed: scaffold the FastAPI project structure and LangGraph agent graph.